MODULE 3(PL/SQL)

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Scenario 1:

```
DECLARE
  CURSOR cust_cursor IS
    SELECT customer_id, age, loan_interest_rate
    FROM customers;

BEGIN
  FOR cust IN cust_cursor LOOP
    IF cust.age > 60 THEN
      UPDATE customers
      SET loan_interest_rate = loan_interest_rate - 0.01
      WHERE customer_id = cust.customer_id;
    END IF;
  END LOOP;
```

```
    COMMIT;
END;
```

Scenario 2:

```
DECLARE
    CURSOR cust_cursor IS
        SELECT customer_id, balance
        FROM customers;

BEGIN
    FOR cust IN cust_cursor LOOP
        IF cust.balance > 10000 THEN
            UPDATE customers
            SET isVIP = 'TRUE'
            WHERE customer_id = cust.customer_id;
        END IF;
    END LOOP;

    COMMIT;
END;
```

Scenario 3:

```
DECLARE
    CURSOR loan_cursor IS
        SELECT customer_id, loan_id, due_date
        FROM loans
        WHERE due_date BETWEEN SYSDATE AND SYSDATE + 30;

BEGIN
    FOR loan_rec IN loan_cursor LOOP
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Reminder: Customer ' || loan_rec.customer_id ||
            ' has loan ' || loan_rec.loan_id ||
            ' due on ' || TO_CHAR(loan_rec.due_date, 'DD-MON-YYYY')
        );
    END LOOP;
END;
```

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

Scenario 1:

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_account IN NUMBER,
    p_to_account   IN NUMBER,
    p_amount       IN NUMBER
)
IS
```

```
    v_balance NUMBER;
BEGIN
    -- Get sender's balance
    SELECT balance
    INTO v_balance
    FROM accounts
    WHERE account_id = p_from_account;

    -- Check sufficient balance
    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
    END IF;

    -- Debit sender
    UPDATE accounts
    SET balance = balance - p_amount
    WHERE account_id = p_from_account;

    -- Credit receiver
    UPDATE accounts
    SET balance = balance + p_amount
    WHERE account_id = p_to_account;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

EXCEPTION
```

```
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Transfer Failed: ' || SQLERRM);
END;
```

Scenario 2:

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_emp_id      IN NUMBER,
    p_percentage  IN NUMBER
)
IS
BEGIN
    UPDATE employees
    SET salary = salary + (salary * p_percentage / 100)
    WHERE employee_id = p_emp_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Employee ID does not exist');
    END IF;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

```
END;
```

Scenario 3:

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
   p_customer_id IN NUMBER,
   p_name       IN VARCHAR2,
   p_balance    IN NUMBER
)
IS
BEGIN
   INSERT INTO customers(customer_id, customer_name, balance)
   VALUES (p_customer_id, p_name, p_balance);

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Customer added successfully.');

EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists.');

   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Scenario 1:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
   UPDATE accounts
   SET balance = balance + (balance * 0.01)
   WHERE account_type = 'SAVINGS';

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');

EXCEPTION
   WHEN OTHERS THEN
```

```
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Scenario 2:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department_id IN NUMBER,
    p_bonus_percent IN NUMBER
)
IS
BEGIN
    UPDATE employees
    SET salary = salary + (salary * p_bonus_percent / 100)
    WHERE department_id = p_department_id;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Scenario 3:

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
```

```
    p_to_account   IN NUMBER,
    p_amount       IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    -- Get source account balance
    SELECT balance
    INTO v_balance
    FROM accounts
    WHERE account_id = p_from_account;

    -- Check sufficient balance
    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001,'Insufficient Balance');
    END IF;

    -- Debit source account
    UPDATE accounts
    SET balance = balance - p_amount
    WHERE account_id = p_from_account;

    -- Credit destination account
    UPDATE accounts
    SET balance = balance + p_amount
    WHERE account_id = p_to_account;

    COMMIT;
```

```
    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Transfer Failed: ' || SQLERRM);
END;
```